操作系统原理课程设计

Experiment Design with Operating System Principle

- 主讲人：邵文华
- 计算机学院（国家示范性软件学院）



课程教学目标

- ✓ **加深**和巩固学生对所学操作系统各个理论和算法知识的理解，以及**综合**运用能力
- ✓ 提高学生利用操作系统知识**分析**问题、**解决**问题能力



课程具体教学内容

- ✓ 针对涉及的核心概念、方法，结合生产、生活中的**实际问题**开展一款**较为完整**的课程软件设计、实现并展示。
  - ✓ 例如教学活动管理（课表/课程/上下课管理），涉及角色（进程/线程），还可以加上多媒体信息...
- ✓ 基本内容：
  - ➢ 进程调度、线程调度、同步/互斥处理、消息传递、文件处理、文件锁...
- ✓ 其他发散一些的内容：
  - ➢ A进程所做的工作在其他终端上显示、**图形界面**...

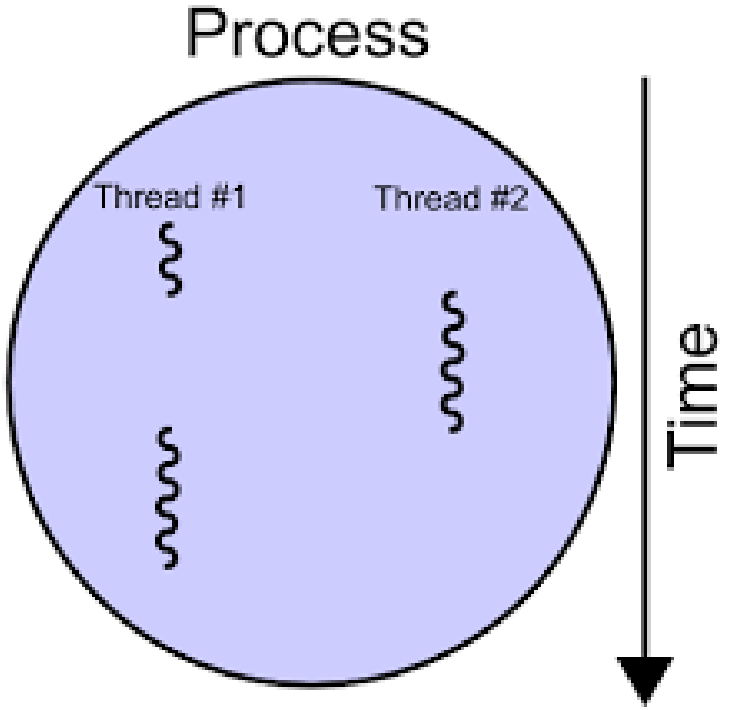
课程可能涉及的操作系统基本原理（1）

- 进程控制和管理
  - ✓ 进程环境
  - ✓ 进程控制
  - ✓ 进程间关系
  - ✓ 守护进程
- 信号和同步处理
  - ✓ 信号处理
  - ✓ 可靠信号
  - ✓ 信号处理程序



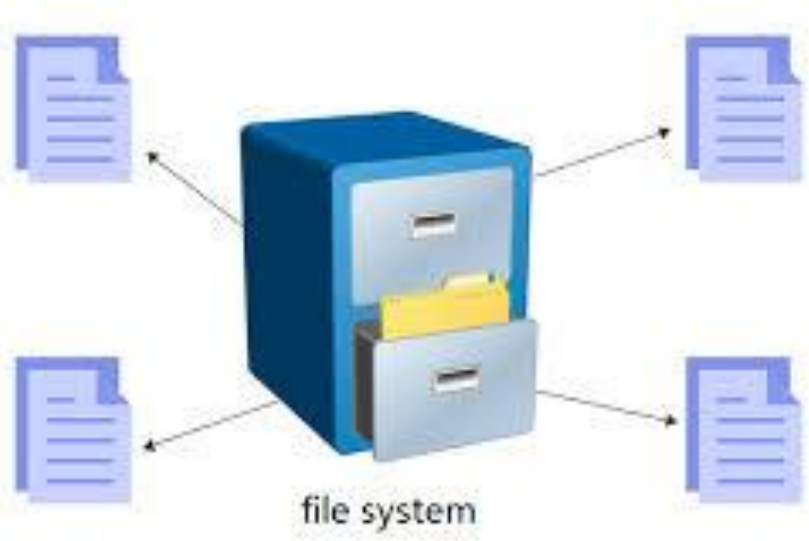
课程可能涉及的操作系统基本原理（2）

- 进程间通信
  - ✓ 管道
  - ✓ 消息队列
  - ✓ 信号量
  - ✓ 共享内存
- 线程及线程编程
  - ✓ 线程创建及管理
  - ✓ 线程同步
  - ✓ 线程间通信



课程可能涉及的操作系统基本原理（3）

- 文件系统
  - ✓ 文件I/O
  - ✓ 文件和目录
  - ✓ 文件流I/O
  - ✓ 系统数据文件和信息
  - ✓ 文件锁机制
- API参考文档



所实现系统必须有守护进程

- 守护进程，顾名思义，也就是专门**守护**一个进程的进程。
- 守护进程的**职责**就是专门**确保**被指定的进程的运行。
- 守护进程也称精灵进程（Daemon），是运行在**后台**的一种特殊进程。
- 它**独立**于控制终端，并且**周期**性的执行某种任务或等待处理某些发生的事件。
- 守护进程是一种**很有用**的进程，linux的大多数服务器就是用守护进程实现的。
  - 比如：Internet服务器inetd。Wed服务器httpd等。同时，守护进程完成了许多系统任务。比如，作业规划进程crond等。



所实现系统必须有守护进程

- 最直接的应用就是**重启**进程。
  - 如果我们的软件崩溃了，为了让软件重新运行起来，软件本身是不方便或者无法做到的。那么可以借助一个专门的进程来帮助自己启动。
- 守护进程当然不仅限于重启进程。
  - 可以守护进程不被关闭，重启进程
  - 也可以守护进程的网络，
  - 必要的配置文件等等。
- 守护进程好比是**保镖**，被守护的进程就是保镖保护的对象。守护进程在很多地方都会有应用。

所实现系统必须有守护进程

- 守护进程的创建流程
  - (1) 执行一个 fork () 之后**父**进程**退出**，子进程继续执行。
  - (2) 子进程调用 setsid( ) 开启一个**新会话**
    - 将子进程与父进程分离，使其成为一个独立的进程组，不再受到父进程的控制
  - (3) 清除进程的 umask 以确保当守护进程创建文件和目录时拥有所需的权限。
  - (4) 修改进程的当前工作目录，通常会改为根目录(/)。
  - (5) 关闭守护进程从其父进程继承而来的所有打开着的文件描述符
  - (6) 在关闭了文件描述符0、1、2之后，守护进程通常会打开/dev/null并使用dup2( )使所有这些描符指向这个设备.
    - 这样做的含义是将标准输入、标准输出和标准错误输出都重定向到/dev/null设备上，也就是将它们丢弃掉，不再输出到终端或其他地方。这样可以防止守护进程产生任何输出，确保其在后台默默运行，不影响终端或其他程序的正常操作。
  - (7) 核心业务逻辑。

所实现系统必须有守护进程

```
/*
    写一个守护进程，每隔2s获取一下系统时间，将这个时间写入到磁盘文件中。
*/

#include <stdio.h>      //标准输入输出库，用于文件操作和打印信息。
#include <sys/stat.h>   //用于定义文件状态和数据类型。
#include <sys/types.h>  //用于定义文件状态和数据类型。
#include <unistd.h>     //提供对 POSIX 操作系统 API 的访问，包括文件操作和进程控制。
#include <fcntl.h>      //用于文件控制操作。
#include <sys/time.h>   //用于获取当前时间。
#include <signal.h>     //用于处理信号。
#include <time.h>       //用于时间处理。
#include <stdlib.h>     //用于标准库处理。
#include <string.h>     //用于字符串处理。
```

所实现系统必须有守护进程

```
// 捕捉到信号之后，获取系统时间，写入磁盘文件
void work(int num) {
        time_t tm = time(NULL);              // 获取当前的系统时间
        struct tm * loc = localtime(&tm); //将 time_t 类型的时间转换为 tm 结构体
        char * str = asctime(loc);           //将 tm 结构体中的时间信息转换为字符串格式
        int fd = open("time.txt", O_RDWR | O_CREAT | O_APPEND, 0664);//打开一个名为 time.txt 的文件
        write(fd ,str, strlen(str));         //将字符串 str（即当前时间）写入到打开的文件中
        close(fd);                           //关闭打开的文件描述符 fd，以释放系统资源。
}
```

所实现系统必须有守护进程

```
int main() {
    // 1.创建子进程，退出父进程
    pid_t pid = fork();

    if(pid > 0) {
        exit(0);
    }
    // 2.将子进程重新创建一个会话
    setsid();
    // 3.设置掩码
    umask(022);
    // 4.更改工作目录
    chdir("/home/nowcoder/");
    // 5. 关闭、重定向文件描述符
    int fd = open("/dev/null", O_RDWR);
    dup2(fd, STDIN_FILENO);
    dup2(fd, STDOUT_FILENO);
    dup2(fd, STDERR_FILENO);
    // 6.业务逻辑
    // 捕捉定时信号
    struct sigaction act;
    act.sa_flags = 0;
    act.sa_handler = work;
    sigemptyset(&act.sa_mask);
    sigaction(SIGALRM, &act, NULL);
    struct itimerval val;
    val.it_value.tv_sec = 2;
    val.it_value.tv_usec = 0;
    val.it_interval.tv_sec = 2;
    val.it_interval.tv_usec = 0;
    // 创建定时器
    setitimer(ITIMER_REAL, &val, NULL);
    // 不让进程结束
    while(1) {
        sleep(10);
    }
    return 0;
}
```

课程进度安排

周次	学时	主要任务
课程第1周	3	讲课1学时、组队(4-6人/组)，小组分工、熟悉API，实验构思
课程第2周	3	课设需求分析，概要设计
课程第3周	3	课堂实验设计小组汇报（**小组PPT**，针对已选题目报告需要做的事情，可能的难点。使用供UML工具、原型工具及思维导图。提交到云邮平台）
课程第4周	3	设计编码实现（集中编程）
课程第5周	3	设计编码实现（集中编程）
课程第6周	3	小组展示可运行系统、交流、验收（代码提交到云邮平台）
课程第7周	3	**个人**整理撰写汇报**PPT**。
课程第8周	3	个人成果汇报答辩（**个人答辩**，提交到云邮平台）

课程考核机制

- 总成绩 = 平时成绩（10%）+课堂实验设计（15%）+ 实验运行（15%）+ 答辩（60%）

✓ 课堂实验设计（15%），小组共同完成，得分项如下，等权：

- 需求调研（阐述系统研发目标，项目必要性）；
- 概要设计（同时分析运用了OS的哪些原理）；
- 小组分工安排。

课程考核机制

- ✓ 实验（15%）：小组共同完成，得分项如下，等权：
  - 可运行：代码要跑通，守护线程有效；
  - 功能完整性：系统是否实现了设计要求中规定的所有功能，并且功能正常运行。评分标准可以是系统功能覆盖率、功能是否符合需求等；
  - 用户体验：系统是否易于使用、界面友好、操作流畅等方面；
  - 性能优化：系统是否具有良好的性能，包括响应速度、资源利用率等方面；
  - 可靠性：系统是否稳定可靠，运行过程中是否存在崩溃、错误等问题；
  - 创新性：系统是否具有创新点、独特性，是否运用了新颖的技术或思想；
  - 各组之间相互观摩学习，相互评价。

评价收集表：https://doc.weixin.qq.com/forms/AK4A5AciAAgABEAYgaKABcPfgd7JgTR3f?page=1

课程考核机制

- ✓ 答辩（60%）：**个人独立**完成，得分项如下，等权：
  - 个人对需求的分析理解（建议通过用例图等工具）；
  - 详细设计（建议时序图等工具）；
  - 系统运行结果分析（发现了哪些问题，如何改进的）；
  - OS基本原理应用充分性（如何应用OS解决实际问题的）；
  - 立意新颖性（分析选题及实现的价值与创新点是什么）。
  - 重要参考：代码上传到gitee，commit质量

处理机调度算法应用示例：医院急诊室

- 医院急诊室类比为操作系统
- 病人类比为进程
- 医生类比为处理机
- 医生的调度策略类比为动态优先级高者优先进程调度算法
- 注意有些病人去做检查，中间可能产生碎片的可用时间



内存管理应用示例：零售库存管理

- 当新货物到达时，系统查找最接近需求量的货架空间
- 如果找到的货架空间大于货物量，将货物放入该货架并更新库存
- 如果货架空间略大于货物量，可以将货物放置在该货架并记录剩余空间
- 如果没有足够的货架空间，系统可以动态调整货架分区以适应新货物的存放



进程同步应用示例：网联车高效率运行问题

- 假设城市车辆已经联网且已具备自动驾驶能力
- 设计一个调度软件能够最大限度提高城市交通通行能力



磁盘调度应用示例：快递配送中心

- 使用类似磁盘调度算法的思想来安排配送路线
- 示例算法：最短路径优先（类似于磁盘调度中的SSTF算法）
- 根据订单目的地的距离，选择距离最近的订单进行配送
- 然后选择离当前配送点最近的下一个目的地进行配送，依次类推，直至所有订单配送完成



下次课上课地点

- 已申请调至实验楼308

授课老师介绍



邵文华
副教授
硕士生导师
shaowenhua@bupt.edu.cn

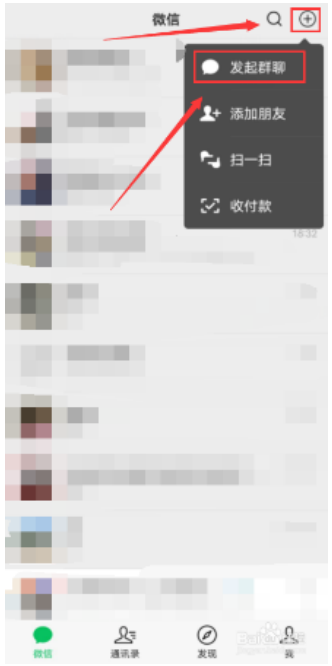
长期致力于多源融合智能移动位置感知技术研究。发表高水平论文23篇，一作中科院一区Top论文2篇(TII 影响因子10.2,IoT 影响因子9.4)，二区Top 1篇，CCF-C会议2篇，总被引252次。申请专利7项（已授权5项），制定国家标准2项（已实施），主持及参与科研课题20余项。指导学生发表SCI论文2篇，EI论文11篇，获优秀硕士学位论文1项。获卫星导航定位科技进步一等奖（一级学会，排名10）等奖励5项。主持开发全时空多源融合智能定位云平台，已应用于华为、联通及三星等公司，部署到央视大楼、广交会，南非平安园区等10余个国家的1000多个场馆，产生经济效益上千万元。

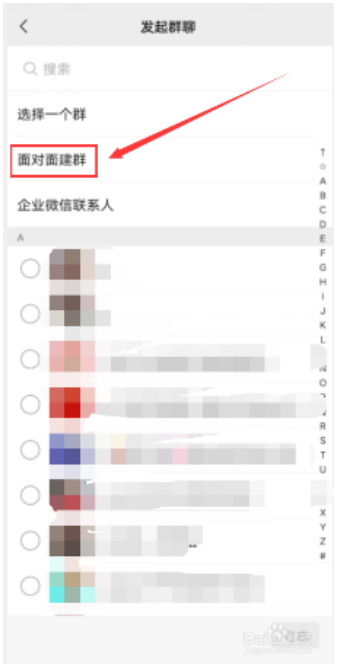
助教

课代表

面对面建群

- 1，发邮件shaowenhua@bupt.edu.cn（邵文华），
- 2，微信群（面对面建群，数字6688）









北京郵電大學
Beijing University of Posts and Telecommunications

谢谢！

邵文华：
shaowenhua@bupt.edu.cn